API (APPLICAITON PROGRAMABLE INTERFACE)

Why do we need an api

suppose a customer opens the FoodGo mobile application

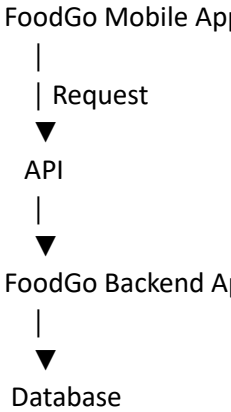
they click on pizza

The mobile application needs information from the backend:

- pizza
- price
- restaurant
- rating
- availability
- offers

But the mobile application should not directly access the database.

So We have:



The API acts as a communication interface between applications.

So mobile application doesn't need to know:

- How database works
- How Java works
- How Node.js works
- How SQL works
- How MongoDB works

it only need to know:

- Which API?
- What request?
- What response?

What is API means

application programming interface

What is an API Request?

Example:
GET /restaurants
it simply means give me the restaurants

GET /restaurants/101
it simply means give me the restaurant number 101

POST /orders
it simply means create a new order

What is an API Response?

The server send the response

Request:
GET /restaurants/101

Response:

```
{
  "id": 101,
  "name": "Food Palace",
  "rating": 4.5,
  "location": "Pune"
}
```


so:
CLIENT
  |
  | GET /restaurants/101
  ▼
SERVER
  |
  | JSON Response
  ▼
CLIENT

What is HTTP

1. it stands for Hypertext Transfer Protocol
2. It is a protocol used for communication between client and server.

So basically, there are common http methods:

1. GET
2. POST
3. PUT
4. PATCH
5. DELETE

1.GET
GET /restaurants - give me the restaurants

2.POST
POST /orders - used to create a new order

```
{
  "restaurantId": 101,
  "items": [
    {
      "name": "Pizza",
      "quantity": 2
    }
  ]
}
```

3.PUT
PUT /users/101 - used to replace an existing resource

```
{
  "name": "Mayur",
  "email": "mayur@example.com",
  "phone": "9999999999"
}
```

4.PATCH
PATCH /users/101 - used for partial updates

```
{
  "phone": "8888888888"
}
```

Only the phone number changes.

5.DELETE
DELETE /orders/501 - Used for delete something

TYPES OF APIs

1. REST
2. GraphQL
3. WebSocket
4. gRPC
5. SOAP
6. Webhooks

REST API (Representational State Transfer)

Rest normally represents data as resources and uses standard HTTP methods.

1. GET (Read data)
2. PUT (Replace something)
3. POST (create something)
4. PATCH (partially update something)
5. DELETE (Delete something)

A REST API generally thinks in terms of resources.

For Example:

1. User
2. Restaurant
3. Order
4. Driver
5. Payment

Endpoints represent those resources.

1. /users
2. /restaurants
3. /orders
4. /drivers
5. /payments

Think:
Resource collection

1. /orders

One resource

2. /orders/501

REST CRUD

Now connect HTTP methods with CRUD.

- C → Create (POST)
- R → Read (GET)
- U → Update (PUT/PATCH)
- D → Delete (DELETE)

FoodGo Example:

CREATE
POST /orders

READ
GET /orders/501

UPDATE
PATCH /orders/501

DELETE
DELETE /orders/501

URL/URI ENDPOINTS

https://api.foodgo.com/restaurants/101

https = protocol
api.foodgo.com = Host
/restaurants/101 = path

REST JSON
REST APIs very commonly exchange JSON.

json - JavaScript Object Notation

Example:
```
{
  "orderId": 501,
  "customer": "Mayur",
  "restaurant": "Food Palace",
  "total": 450
}
```

understand:
Key Value

orderId 501
customer Mayur
total 450

HTTP STATUS CODES

200 ok
Request successful.
GET /restaurants

Response: 200 ok

201 - created
A new resource was created.
POST /orders
Response: 201 created

204 - No Content
Request succeeded but there is no response body.
DELETE /orders/501
Response: 204 no content

400 - Bad Request
The request is invalid.
POST /orders
Response: 400 bad request

401 - Unauthorized
authentication is missing/invalid (Authorization token missing)
401 unauthorized

403 - Forbidden
User is authenticated but not allowed to perform the action.
403 Forbidden

404 - Not Found
GET /orders/99999
If order doesn't exist
404 Not Found

409 - Conflict
The request conflicts with the current state.(with an email that already exists.)
POST /users
409 Conflict

500 - Internal Server Error
problems on the server side
500 internal server error

502 - Bad Gateway
A gateway/proxy received an invalid response from an upstream server.

503 Service Unavailable
The service is currently unavailable.
Possible causes:
- Server overloaded
- Service down
- Maintenance

- Kubernetes pods unavailable

REST API EXAMPLES

1.customer opens FoodGo
GET /restaurants

Response:

```
[
 {
  "id": 101,
  "name": "Food Palace"
 },
 {
  "id": 102,
  "name": "Pizza Hub"
 }
]
```

2.Customer selects restaurant
GET /restaurants/101/menu

3.Customer creates order
POST /orders

```
{
 "restaurantId": 101,
 "items": [
  {
   "itemId": 10,
   "quantity": 2
  }
 ]
}
```

4.server response

```
{
 "orderId": 501,
 "status": "PLACED",
 "total": 450
}
```

FOODGO AUTHENTICATION

1.Customer logs in
POST /auth/login

2.request

```
{
```

```
  "email": "mayur@example.com",
  "password": "password"
}
```

3.Response might contain a toke

```
{
  "accessToken": "eyhfwuvrghrg"
}
```

4.Then whenever u get the request
GET /orders
Authorization: Bearer "eyhfwuvrghrg"

5.Backend verifies the token and creates the communication with database

RATE LIMITING

Suppose someone sends:
1,000,000 requests

POST /login
That can cause abuse or overload.

Rate limiting can restrict requests.

Example:
100 requests/minute/IP

After the limit:
429 Too Many Requests

PAGINATION

Imagine FoodGo has:
10 million restaurants

Don't return everything in one response.

Use pagination:
GET /restaurants?page=1&limit=20

Response:

```
{
  "page": 1,
  "limit": 20,
  "total": 1000,
```

```
  "data": [
    {}
  ]
}
```

NOW THE PROBLEMS WITH THE REST

Suppose the FoodGo customer dashboard wants:
Customer name
Last 3 orders
Reward points
Saved addresses
Recommended restaurants

With REST, the client might call several endpoints:
GET /users/10
GET /users/10/orders
GET /users/10/rewards
GET /users/10/addresses
GET /users/10/recommendations

Potentially this means multiple network requests.

GraphQL

1) GraphQL is a query language for APIs and a runtime for executing those queries.
2) Instead of calling many resource-specific endpoints, a client can request a particular data shape through a GraphQL API.

Typical endpoint:
POST /graphql

Example:

```
query {
  user(id: 123) {
    name
    photo
  }
}
```

The client asks specifically for:

1) name
2) photo

The response can contain those requested fields.

Components of GraphQL

1.Query - (GET - to fetch data)

```
query {
  restaurant(id: 101) {
    name
    rating
  }
}
```

2.Mutation - (POST/PUT/DELETE - To Change data)

```
mutation {
  createOrder(...) {
    id
    status
  }
}
```

3.Subscription - (Receive real time updates)

```
subscription {
  orderStatusChanged {
    orderId
    status
  }
}
```

Subscriptions are commonly implemented using persistent real-time transport such as WebSockets.

WEBSOCKET (LIVE DELIVERY TRACKING)

REST is normally request-response:
Client → Request
Server → Response

But some applications need continuous communication.
Example:

1. Chat
2. Live delivery tracking
3. Online gaming
4. Live dashboards
5. Trading systems
6. Real-time notifications

FoodGo example:
Driver location:
18.5204, 73.8567

The customer shouldn't have to repeatedly ask:
Where is driver?
Where is driver?
Where is driver?
Where is driver?

A WebSocket connection can allow ongoing communication.

Client Send HTTP Request with upgrade header:
GET /chat HTTP/1.1
upgrade: websocket
connection: upgrade
sec-websocket-key: ABCEFGHID=

Server Agrees
HTTP/1.1 101 switching protocol
upgrade: websocket
connection: upgrade

WEBSOCKETS STATES

1. connecting
2. open
3. closing
4. closed

	state	value	meaning
1)	state	value	meaning
2)	connecting	0	handshake/connection being established
3)	open	1	Connection established
4)	closing	2	Connection is being closed
5)	closed	3	Connection has closed

GRPC (GOOGLE REMOTE PROCEDURE CALL)

1. Used for (COMMUNICATION BETWEEN MICROSERVICES)
2. gRPC is commonly used for communication between services, especially in microservice architectures.

FoodGo:

1. order service
2. payment service

3. restaurant service
4. delivery service
5. notification service

Rest we used but rest api can send the requests in json and it will take time to process while

JSON (REST):
{
  orderID: 12345
  amount: 450
  status: "confirmed"
}

grpc send the requests in binary and it enables the fast communication between the microservices

PROTOBUF (gRPC):
Binary encoded:
08 b9 60 10 a9 05 1a
3-10x smaller than JSON
5-7x faster to parse
strictly typed - no surprises

Performance depends on workload, implementation, network, serialization, payload size, and many other factors.

Advantages can include:

1. Compact messages
2. Efficient serialization
3. Strong schemas
4. Code generation
5. HTTP/2 transport
6. Streaming support

gRPC Communication Types:

1. Unary
   (one request one response)

2. server streaming
   (one request multiples responses)
   Example: Live order tracking

3. client streaming
   (many request one response)
   Example: File uploading in chunks

4. Bidirectional streaming
   (many requests many responses)

gRPC and HTTP/2

gRPC commonly uses HTTP/2.

HTTP/2 provides features such as:

1. Multiplexing
2. Header compression
3. Binary framing
4. Stream-based communication

Multiplexing allows multiple streams to share a single TCP connection.

SOAP (SIMPLE OBJECT ACCESS PROTOCOL)

1. Used in Older banking system
2. a strict XML based protocol used for secure and reliable communication between applications.

It traditionally uses XML.
Example:

```
<soap:Envelope>
  <soap:Body>
    ...
  </soap:Body>
</soap:Envelope>
```

SOAP is common in some enterprise and legacy systems, including certain banking, insurance, and government integrations.

SOAP Security
SOAP ecosystems support specifications such as WS-Security.

WS-Security can provide mechanisms involving:

1. authentication
2. digital signature
3. encryption

This is one reason SOAP remains present in some enterprise environments.

WEBHOOKS (EVENT BASED HTTP CALLBACK)

1. Webhooks are different from normal request-response APIs.
2. When an event happens, send an HTTP request to a predefined URL.

Example:
FoodGo

|
| Payment request
↓
Payment Provider

Instead of FoodGo repeatedly asking:
Is payment completed?
Is payment completed?
Is payment completed?

The payment provider can call FoodGo's webhook when the event occurs.

FoodGo provides:
POST /webhooks/payment

```
{
 "event": "payment.success",
 "paymentId": "PAY123",
 "orderId": "501",
 "amount": 450
}
```

FoodGo receives it and updates:
Order #501
↓
Payment SUCCESS
↓
Order CONFIRMED

Webhook Security
A webhook endpoint must not simply trust anyone who sends a request.

Common protection mechanisms include:

1. HTTPS
2. Signature verification
3. Timestamp validation
4. Secret/shared key
5. IP restrictions where appropriate
6. Replay protection
7. Idempotency
